Outline of the lecture

- COMBINATIONAL CIRCUITS
  - HALF ADDER
  - FULL ADDER
  - HALF SUBTRACTOR
  - FULL SUBTRACTOR
  - BINARY PARALLEL ADDER
  - BINARY ADDER-SUBTRACTOR
  - BCD ADDER
  - SERIAL ADDER
  - LOOK AHEAD CARRY ADDER

COMBINATIONAL CIRCUITS

- Combination logic circuits are the circuits whose outputs depend upon on the present input combination
  - The previous inputs or outputs do not alter the present outputs
  - They do not have memory
- They can be represented by Boolean functions
- Combinational circuit comprises of input variables, logic gates and output variables
  - Inputs and outputs are binary in nature i.e. can only take 0 or 1 as the possible values

COMBINATIONAL CIRCUITS

- Designing Procedure for combinational logic circuits will follow the steps as:
  - Input variables and output variables are identified and assigned distinct letter symbols
  - Truth table is derived, which states value of each output for all possible input combinations
  - Simplified Boolean expression is obtained for each output variable, either by K-map or Boolean algebra
  - Logic circuit is drawn using desired logic gates

- Things to remember
  - 'n' input variable will have 2^n input combination, therefore number of rows in truth table will be 2^n
  - For the system having 'n' input variables and 'm' output variables, the number of columns in truth table will be 'n+m'

COMBINATIONAL CIRCUITS

- Combinational circuits that we need to study
  - Code Convertors
  - Adders
  - Subtractors
  - Magnitude Comparators
  - Parity bit Generators
  - Encoders and Decoders
  - Multiplexers and De-multiplexers

HALF ADDER

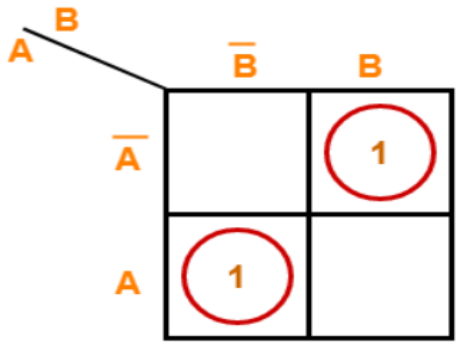
- Combinational circuit that performs arithmetic addition
- Follows the rules of binary addition
- The inputs are two binary bits, addend ‘A’ and augend ‘B’
- The outputs are a sum bit ‘S’ and a carry bit ‘C’

- The truth table for half adder (HA) with two inputs (A,B) and two outputs (S,C) is

Inputs		Outputs	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

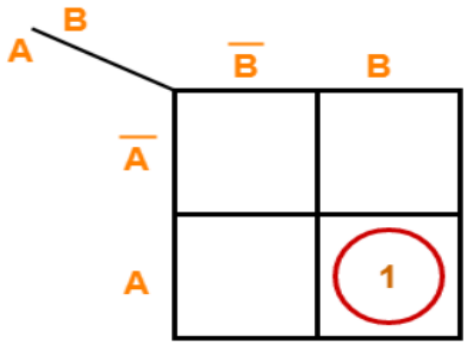
HALF ADDER

For S:



$S = A \oplus B$

For C:

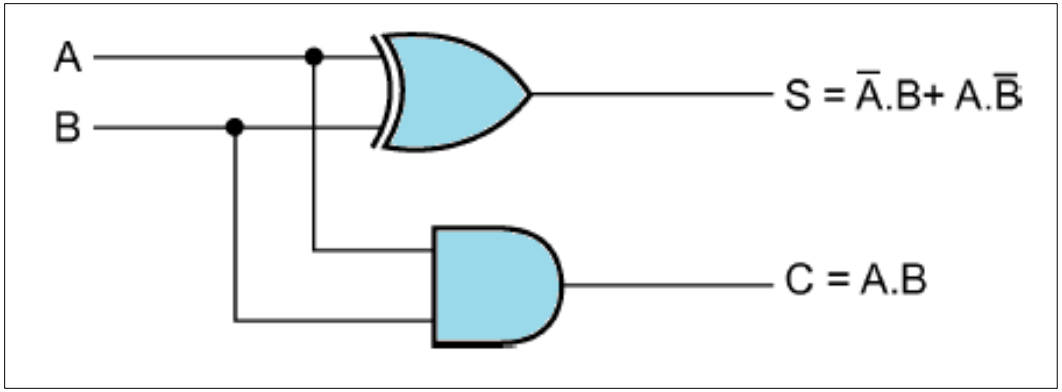


$C = A \cdot B$

- By minimizing using K-map, the reduced relations for Sum and Carry bits are obtained as
  - Sum (S) = A'B +AB' = A (XOR) B
  - Carry (C) = AB

HALF ADDER

- The combinational circuit for half adder can be realized using
  - one XOR gate to calculate Sum (S) and
  - one AND gate to calculate Carry (C)



FULL ADDER

- Combinational circuit that performs arithmetic addition
- Follows the rules of binary addition
- Half adder can add only two bits at a time, no provision to add carry generated from previous bit addition. To resolve this, full adder circuit is designed
- There are three inputs each of one bit: addend 'A', augend 'B' and carry from previous bit addition called carry-in 'C_{in}' [$A + B + C_{in}$]
- The outputs are a sum bit 'S' and a carry bit called carry-out 'C_{out}'.
- This carry bit, C_{out}, is propagated to next significant bit for subsequent addition or provided as the output

FULL ADDER

- The truth table for full adder (FA) with three inputs (A,B, C_{in}) and two outputs (S,C_{out}) is

Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

FULL ADDER

By minimizing using K-map, the reduced relations for Sum and Carry-out bits are obtained as

- Sum (S)

  $= A'B'C_{in} + A'BC_{in}' + AB'C_{in}' + ABC_{in}$

  $= A \text{ (XOR) } B \text{ (XOR) } C_{in}$

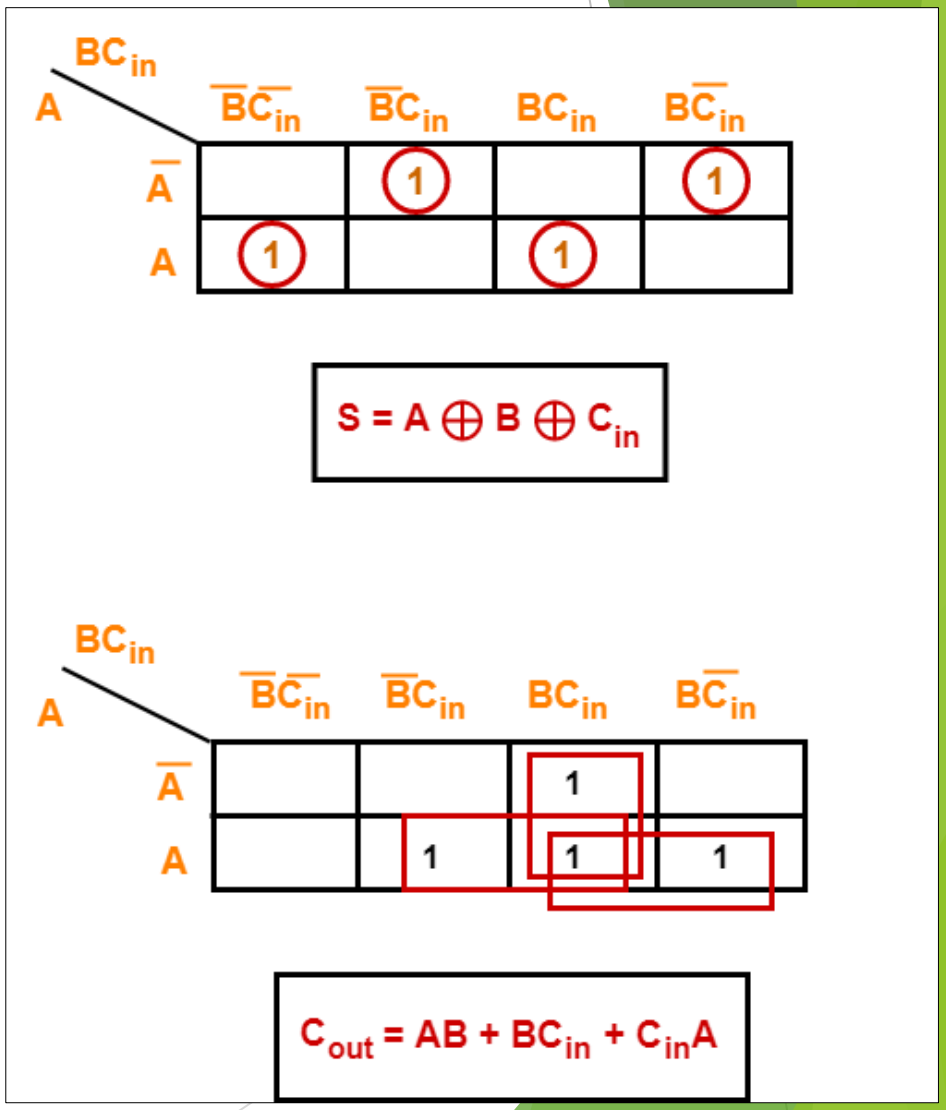
- Carry-out (C_{out})

  $= AB + BC_{in} + AC_{in}$

- Carry-out (C_{out}) can also be written as

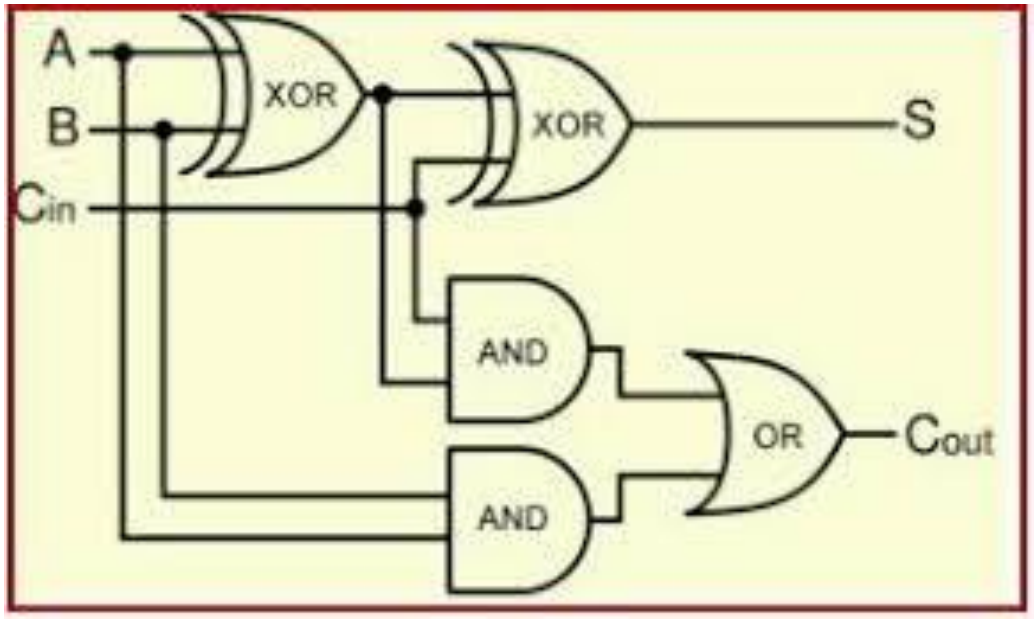
  $= A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in}$

  $= C_{in} [A \text{ (XOR) } B] + AB$



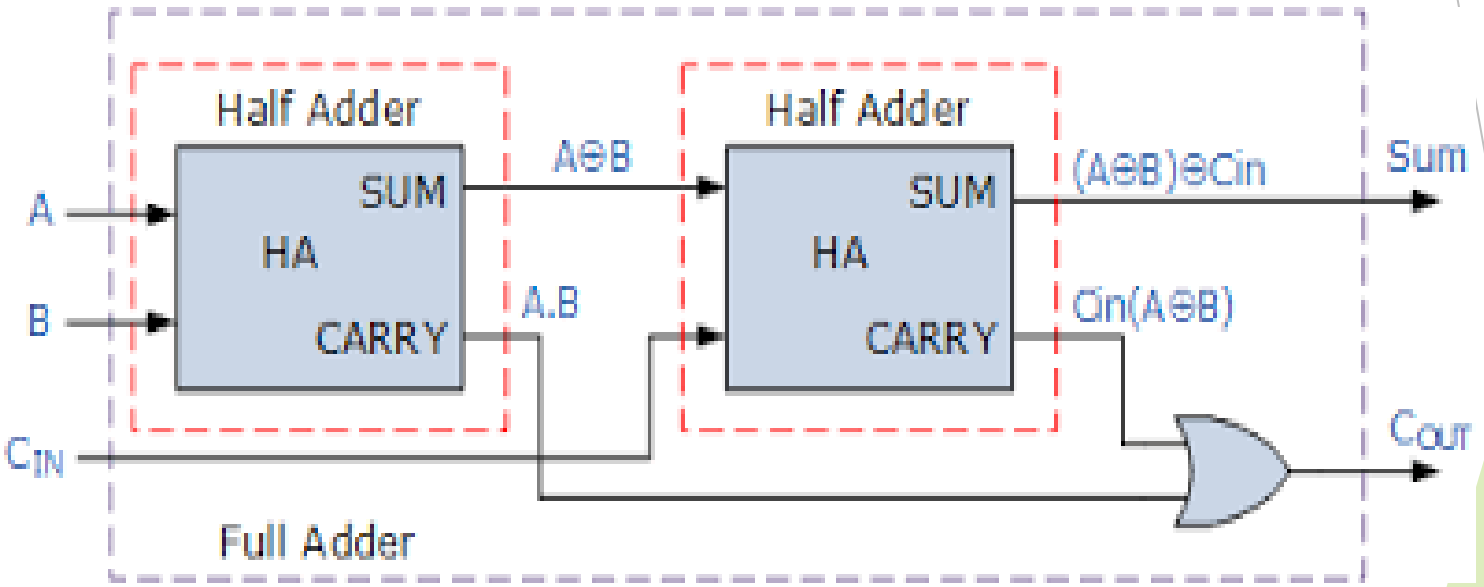
FULL ADDER

- The combinational circuit for full adder can be realized using
  - two XOR gates to calculate Sum (S) and
  - two AND gates and one OR gate to calculate Carry-out (C_{out})



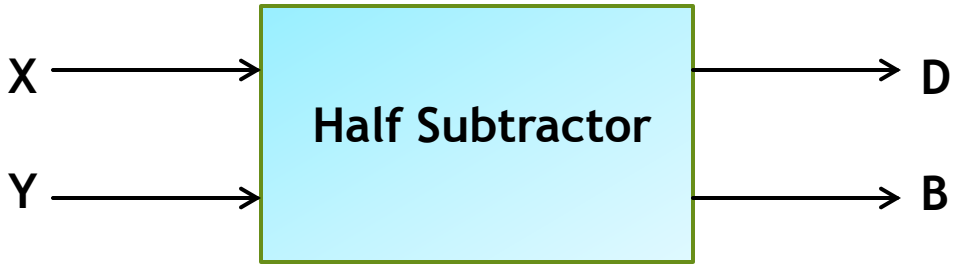
FULL ADDER

- The full adder can also be realized using two half adders and an OR gate



HALF SUBTRACTOR

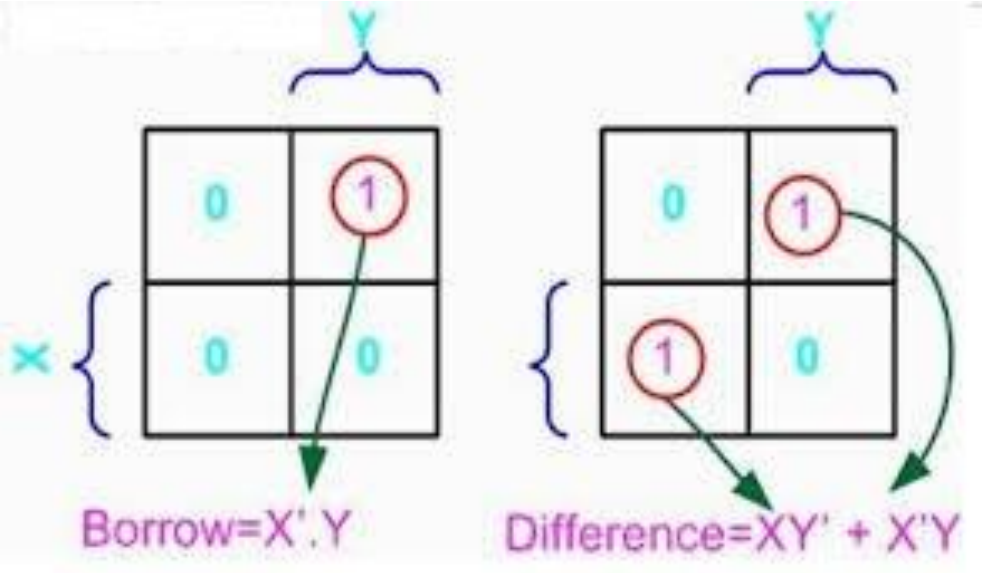
- Combinational circuit that subtracts one bit from another
- Follows the rules of binary subtraction
- The inputs are two binary bits, minuend 'X' and subtrahend 'Y' [X-Y]
- The outputs are a Difference bit 'D' and a Borrow bit 'B'



- The truth table for half subtractor (HS) with two inputs (X,Y) and two outputs (D,B) is

Inputs		Outputs	
X	Y	D	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

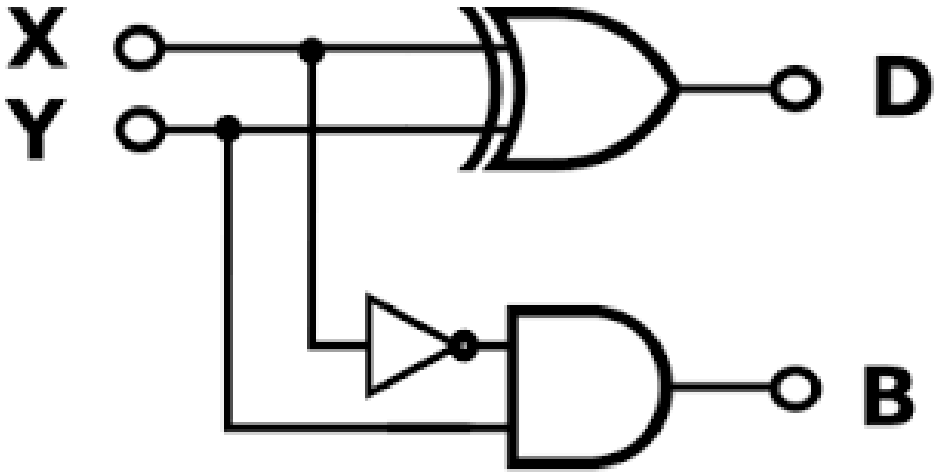
HALF SUBTRACTOR



- By minimizing using K-map, the reduced relations for Difference and Borrow bits are obtained as
  - Difference (D) = X'Y +XY' = X (XOR) Y
  - Borrow (B) = X'.Y

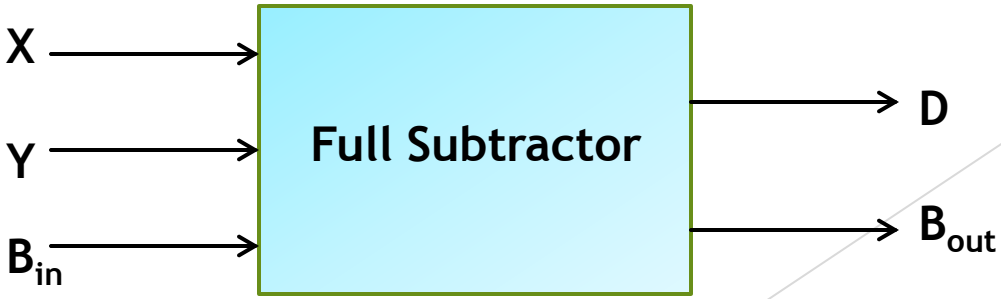
HALF SUBTRACTOR

- The combinational circuit for half subtractor can be realized using
  - one XOR gate to calculate Difference (D) and
  - one AND gate and one NOT gate to calculate Borrow (B)



FULL SUBTRACTOR

- Combinational circuit that performs arithmetic subtraction
- Follows the rules of binary subtraction
- Like half adder, half subtractor can also subtract only two bits at a time. It cannot subtract borrow generated from previous bit subtraction. To resolve this, full subtractor circuit is designed
- There are three inputs each of one bit: minuend 'X', subtrahend 'Y' and borrow from previous bit subtraction called borrow-in 'B_{in}' [$X-Y-B_{in}$]
- The outputs are a Difference bit 'D' and a borrow bit called carry-out 'B_{out}'.
- This borrow bit, B_{out}, is propagated to next significant bit for subsequent subtraction or provided as the output

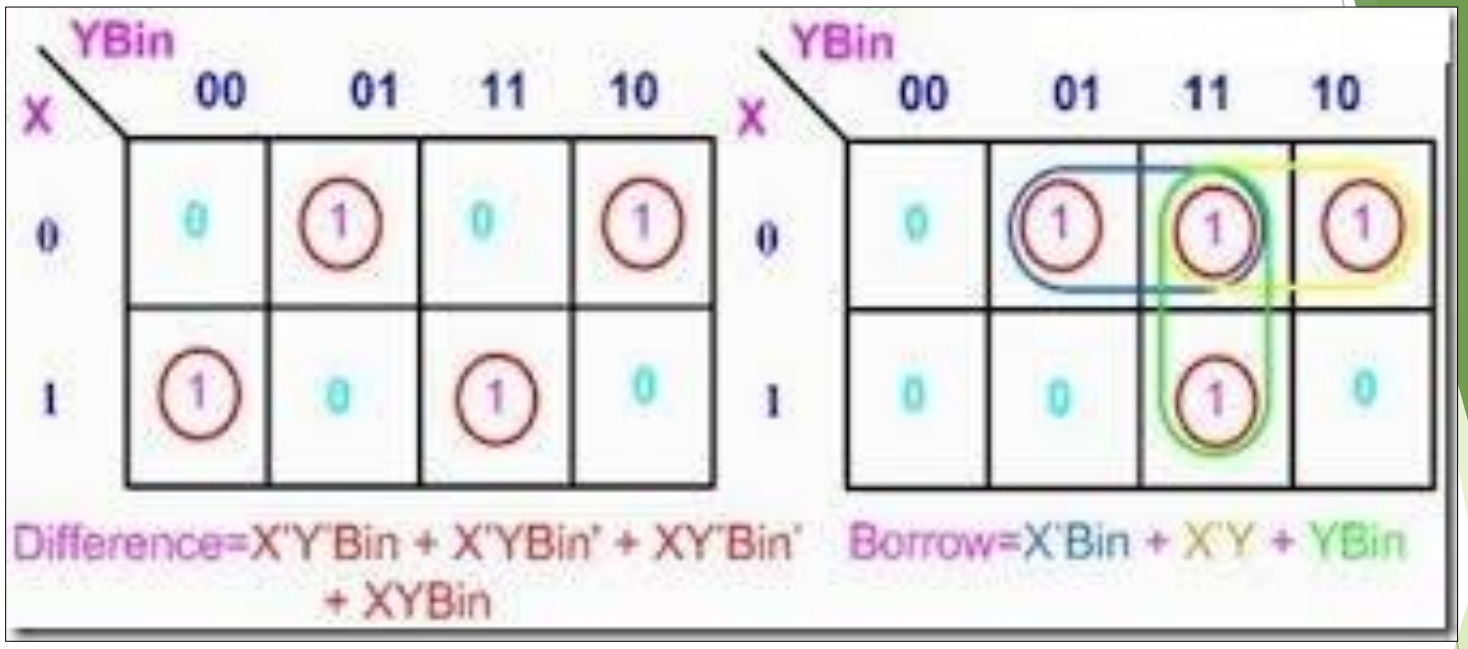


FULL SUBTRACTOR

- The truth table for full subtractor (FS) with three inputs (X,Y, B_{in}) and two outputs (D, B_{out}) is

Inputs			Outputs	
X	Y	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

FULL SUBTRACTOR



Using K-map, the reduced relations for D and B_{out} are obtained as

- **Difference (D) = $X'Y'B_{in} + XYB_{in} + X'YB_{in}' + XY'B_{in}'$**

  = X (XOR) Y (XOR) B_{in}

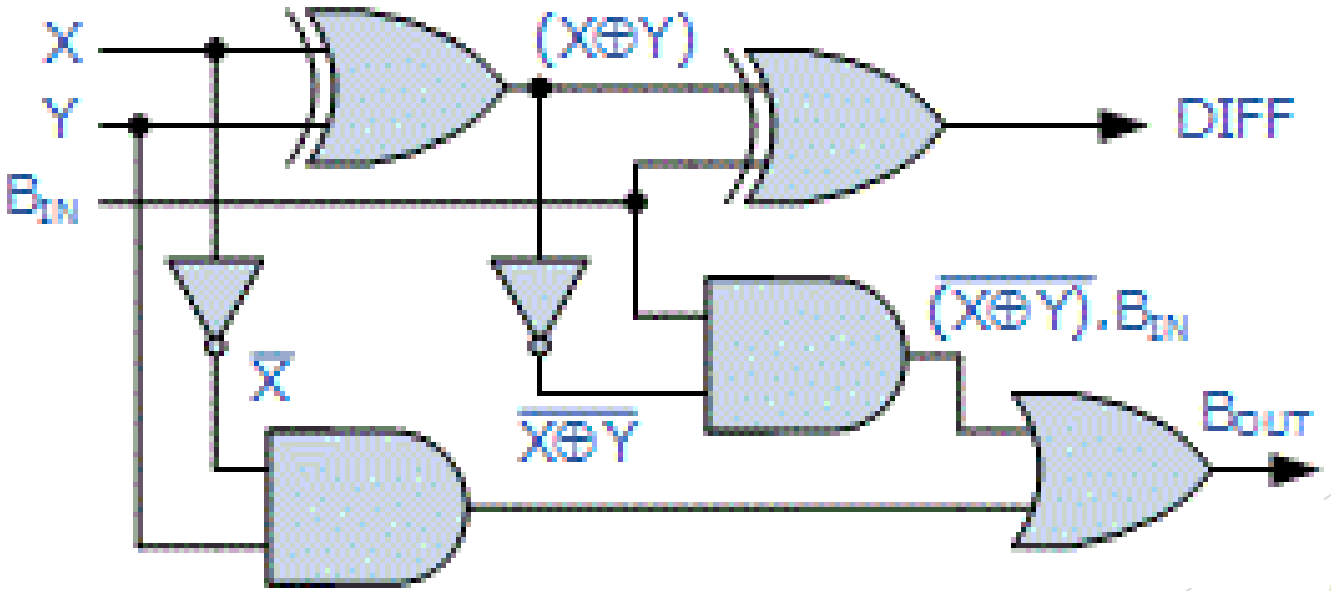
- **Borrow-out(B_{out}) = $X'Y + X'B_{in} + YB_{in}$**
- **Borrow-out (B_{out}) can also be written as**

  = $X'Y'B_{in} + XYB_{in} + X'YB_{in} + X'YB_{in}'$

  = B_{in} [X (XOR) Y]' + X'Y

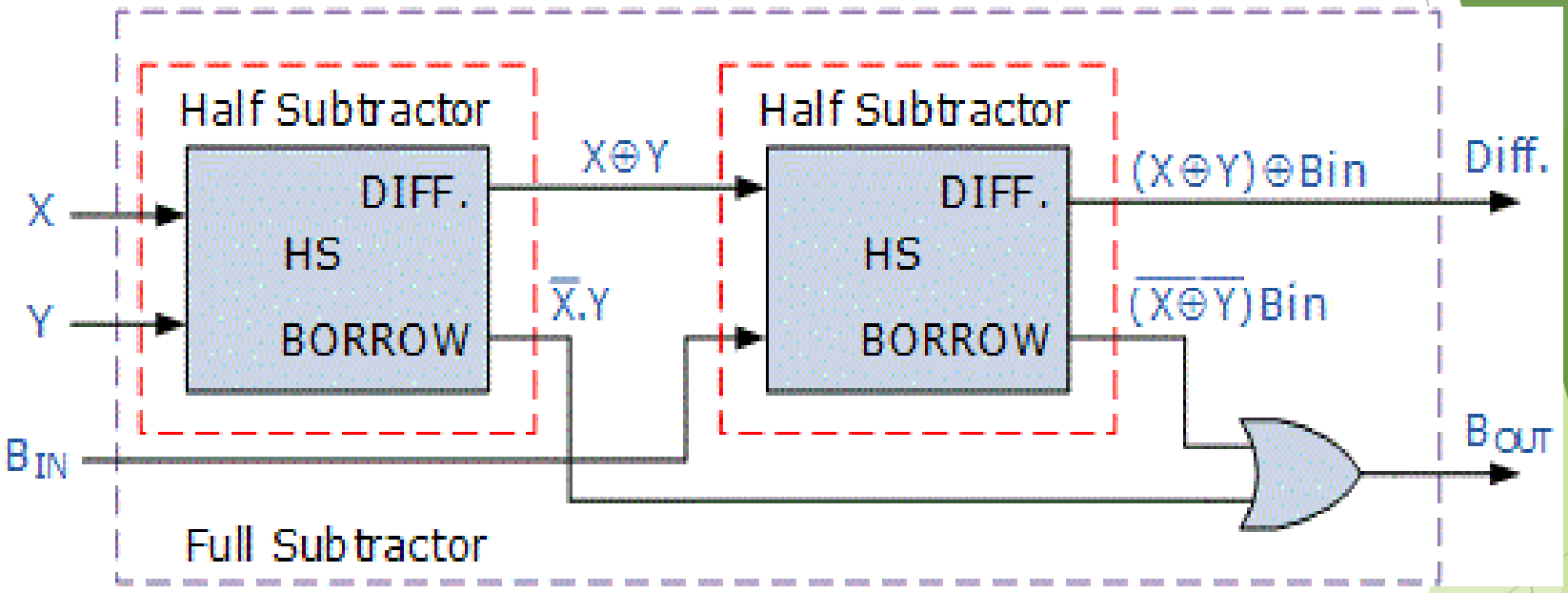
FULL SUBTRACTOR

- The combinational circuit for full subtractor can be realized using
  - two XOR gates to calculate Difference (D) and
  - two AND gates, two NOT gates and one OR gate to calculate Borrow-out (B_{out})



FULL SUBTRACTOR

- The full subtractor (FS) can also be realized using two half subtractors (HS) and an OR gate

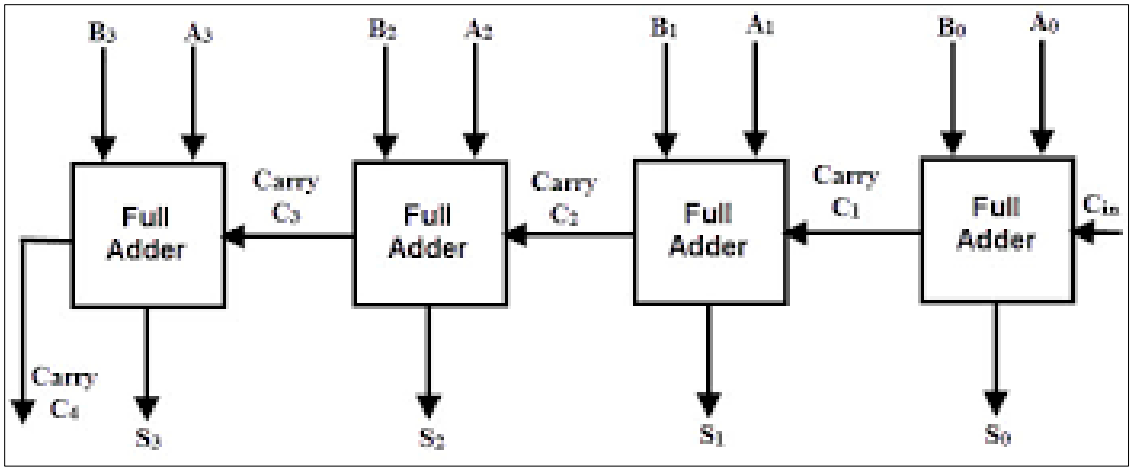


BINARY PARALLEL ADDER

- Binary Parallel adder performs addition of two 'n-bit' binary numbers, addend A and augend B, in parallel
- n-bit addition requires 'n' Full-adders
  - Addition of Least significant bits (LSBs) of A and B can be done by either by a half-adder or a full-adder
  - Full adders are connected in chain
  - The C_{out} of each full-adder is connected to C_{in} of next higher order full-adder
- Addition generates the sum bits as parallel outputs
  - Addition of two 4-bit numbers, $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$, will result in sum $S_3S_2S_1S_0$ and final output carry (generated after addition of A_3 and B_3)
  - Will require 4 Full-adders cascaded such that C_{out} of FA_0 is connected to C_{in} of FA_1 , C_{out} of FA_1 is connected to C_{in} of FA_2 and so on
- Also called as RIPPLE CARRY ADDER

BINARY PARALLEL ADDER

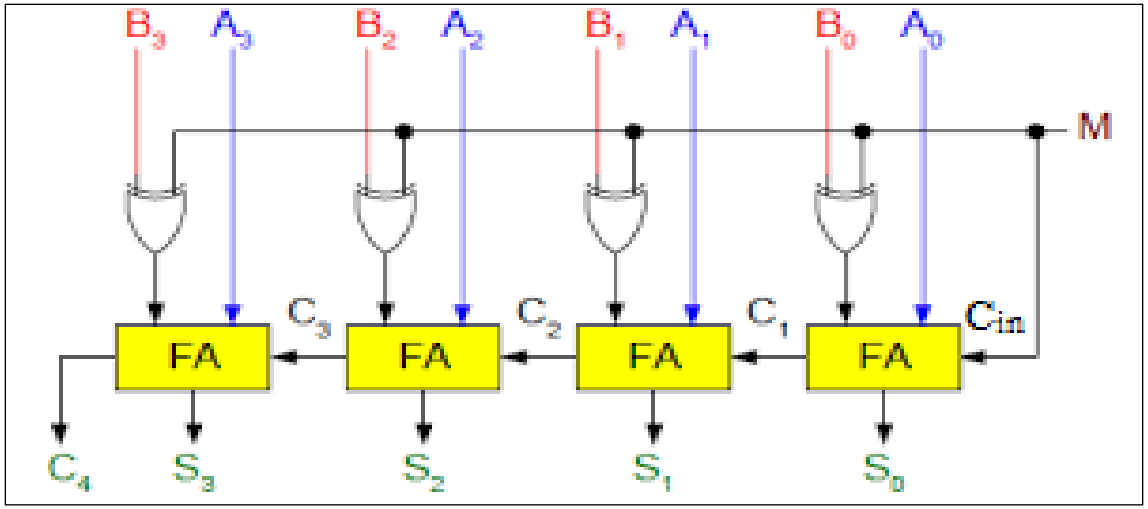
- Addition of two 4-bit numbers, $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$, requires 4 full-adders
- Outputs are 4-bit Sum $S_3S_2S_1S_0$ and a final output carry (C_4) due to addition of bits A_3 and B_3
- All the respective bits of A and B are added simultaneously to generate the sum bit and carry for next bit position



BINARY ADDER—SUBTRACTOR

- Subtraction of two binary numbers can be performed using Binary Parallel Adder
- Adding 2's complement of subtrahend to minuend is equivalent to subtracting subtrahend from minuend
  - A − B = A + (2's complement of B)
- 2's complement of B is obtained using
  - inverters to calculate 1's complement
  - Then adding '1' at LSB through carry-in (C_{in}) of Binary parallel adder
- A common circuit called binary adder—subtractor can perform both addition and subtraction of two binary numbers
  - Exclusive - OR gates are included at input of each full-adder circuit to convert the binary parallel adder into binary adder-subtractor circuit

BINARY ADDER—SUBTRACTOR



- This circuit performs addition or subtraction of two 4-bit numbers, A ($A_3A_2A_1A_0$) and B ($B_3B_2B_1B_0$).Output will be 4-bit Sum ($S_3S_2S_1S_0$) and output carry, C_4
- MODE input 'M' controls the binary operation to be performed. 'B' is Ex-ORed with 'M' and provided as an input to full-adder
- M = 0 → B (XOR) M = B (XOR) 0 = B
  - FAs have inputs A, B, and C_{in} (=0). The circuit works as a Binary Adder
- M = 1 → B (XOR) M = B (XOR) 1 = B'
  - FAs have inputs A, B', and C_{in} (=1)
  - B' and C_{in}=1, together forms 2's complement of B. The circuit works as a Binary Subtractor. [A-B = A+(2's complement of B)]

BCD ADDER

- BCD (Binary Coded Decimal) codes or 8421 codes comprises of 4-bit binary representation for decimal digits 0,1,2,3,4,5,6,7,8 and 9 (0000 to 1001)
- BCD Adder is the combinational circuit that adds two BCD digits and the sum is also a valid BCD number
- The addend and augend are BCD numbers, means each can have any value from 0 (0000) to 9 (1001). Hence, the sum can range only from 0 to 18
- Maximum sum that can be obtained by adding two BCD digits is 19, when there in a carry-in from previous digit addition ($A + B + C_{in} = 9 + 9 + 1$).
- Till the Sum is less than or equal to 9 (1001), it can be represented by corresponding single BCD digit. But, for sum in range of 10 to 19, it is represented by two BCD digits i.e. 10 is (0001 0000);15 is (0001 0101);19 is (0001 1001)

BCD ADDER

- BCD adder circuit comprises of a correction circuit to detect and perform the required conversion of sum from non-valid BCD value to valid BCD value
- The steps for performing BCD addition are
  - Add the two 4-bit BCD digits using binary parallel adder
  - For the positions where the sum is equal to or less than 9, the sum is a valid BCD number and therefore, no correction is required
  - For the positions where the sum is greater than 9 and less than or equal to 19, the sum is a non-valid BCD number. Therefore, the correction is required
- Correction is done by adding binary 6 (0110) to the obtained sum. Adding '6' maps the binary sum to its equivalent valid BCD number. An output carry (C) is also produced for next digit addition
- The necessary condition that should be 'TRUE' or '1' for the correction circuit to operate is calculated from the truth table that maps Binary sum into the equivalent BCD sum

BCD ADDER

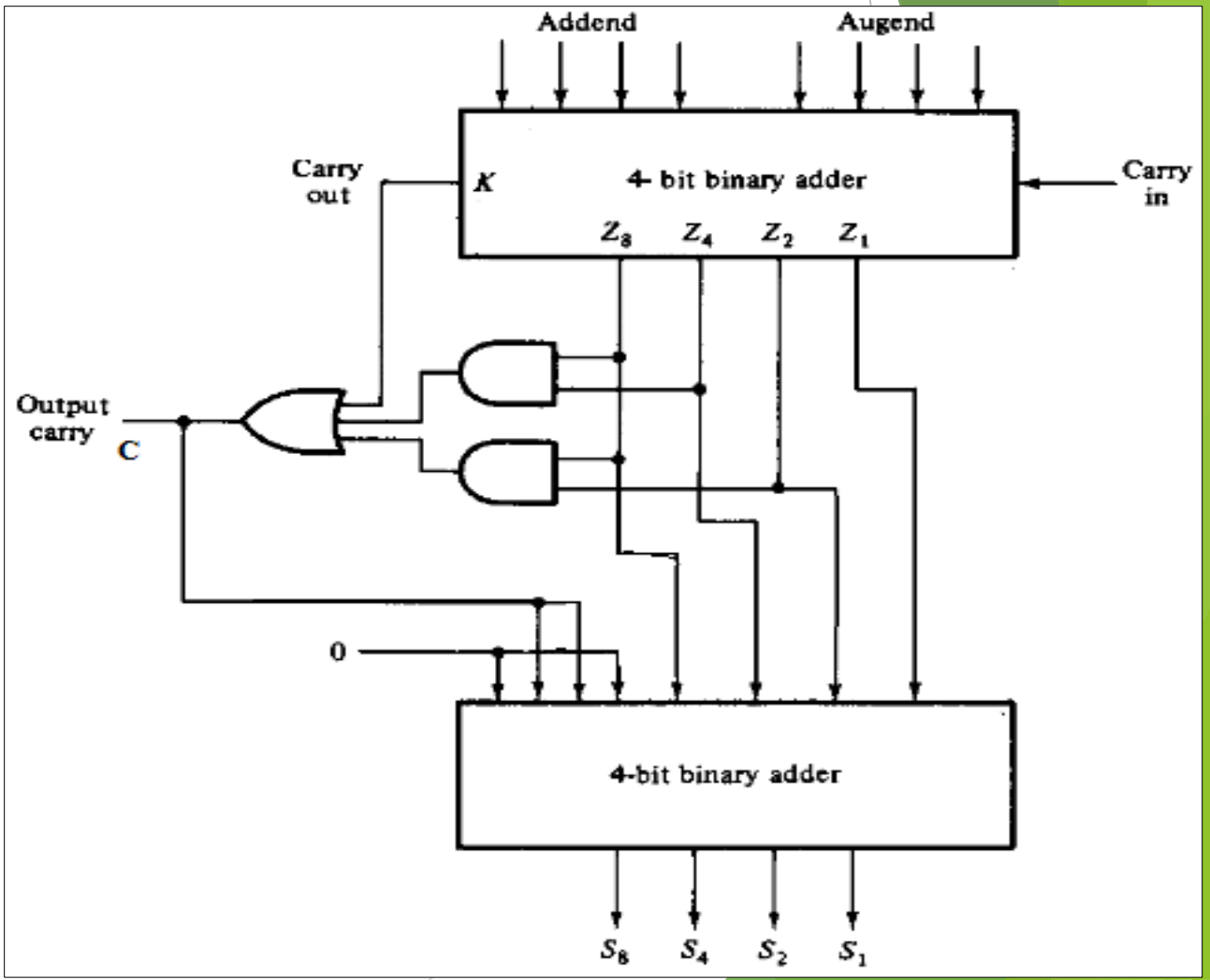
Truth table for mapping Binary sum ($KZ_8Z_4Z_2Z_1$) to BCD sum($CS_8S_4S_2S_1$). The correction is required when

- Output carry in Binary Sum (K) =1
- Z_8 is 1 and either of Z_4 and Z_2 is 1 → $Z_8 Z_4 + Z_8 Z_2 = 1$
- Hence, $C = K + Z_8 Z_4 + Z_8 Z_2$ has to be 1 for 6 (0110) to be added

Binary Sum					BCD Sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

BCD ADDER

- BCD adder comprises of two 4-bit binary adder circuits
- First binary adder adds Addend and Augend, with input carry, C_{in} to generate Binary sum ($KZ_8Z_4Z_2Z_1$)
- Using K, Z_8, Z_4 and Z_2, value of output carry (C) is calculated
- If C = 0, then nothing is added to binary sum. The BCD sum is same as binary sum
- If C = 1, then binary 0110 is added to binary sum through second binary adder. The binary sum is mapped to equivalent BCD sum with output carry (C) as 1

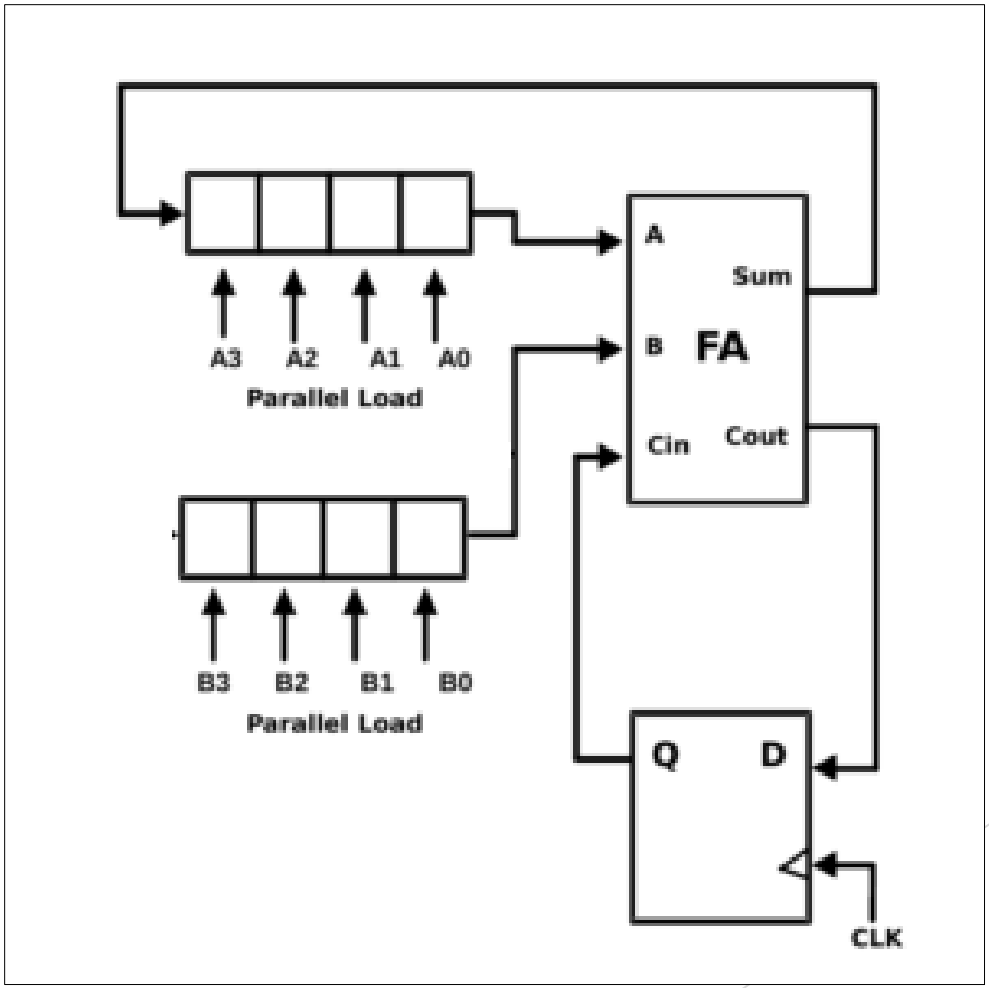


SERIAL ADDER

- Serial Adder adds bits of two binary numbers in serial order
- Only one pair of bits are added at a time, therefore only one Full-adder is required for addition of addend (A) and augend (B)
- Input bits are stored in registers and shifted out, one bit at a time, as inputs to the full adder
- Addition of serial input bits start from the LSBs (A_0, B_0) then (A_1, B_1) and so on till MSBs.
- Output Carry (C_{out}) bit generated from any one addition is saved in a flip-flop and provided as input carry (C_{in}) to next higher order bit addition
- Output sum is stored in a register
- Serial adders are slower than parallel adders, as only one bit is added per clock pulse
- They are used where circuit minimization is more important than speed

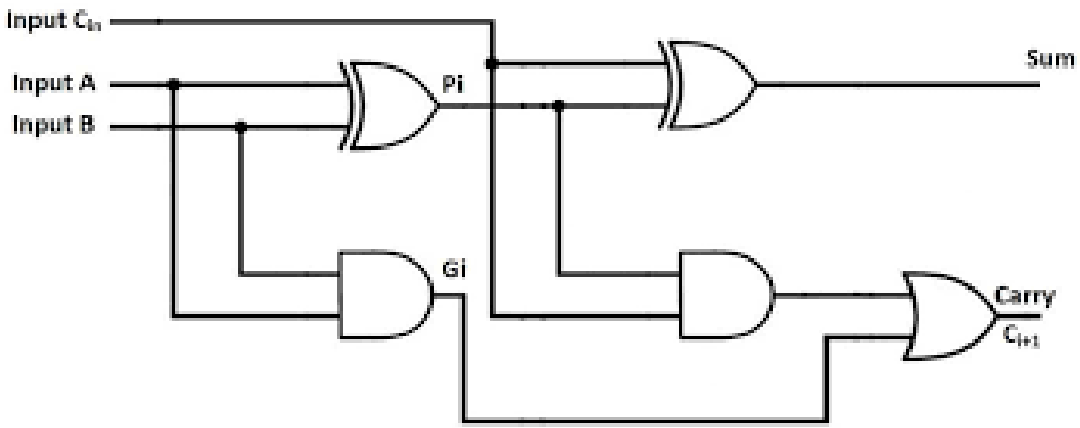
SERIAL ADDER

- Addend (A) and augend (B) are stored in separate registers
- Output sum (S) can either be stored in separate register or in same as any of the input



LOOK AHEAD CARRY ADDER

- In Parallel adder, the speed of addition is controlled by the time required by the carry inputs to propagate through all the FAs
- All the addend and augend bits are already loaded to each FA, but the sum bit can only be calculated after the value of carry-in is propagated and available for addition
- In a 4-bit parallel adder, the total propagation time taken by the carry-out to propagate will be time taken in eight logic gates
- For n-bit parallel adder, propagation time for carry-out is ‘2n’ gate levels, hence, the limiting factor for the speed



LOOK AHEAD CARRY ADDER

- LOOK AHEAD CARRY adder speeds up the addition by reducing the propagation delay time
- All the input bits are examined simultaneously and carry-in bits, for all stages, are also generated simultaneously
- Two additional functions are incorporated in this adder to speed up the addition process
  - Carry Generate(G_i): provides the output carry when both A and B are '1', irrespective of value of carry-in (C_i)

$$G_i = A_i.B_i$$

  - Carry propagate (P_i): function associated with the propagation of carry from C_i to C_{i+1}

$$P_i = A_i \text{ (XOR) } B_i$$

  - Sum and Carry-out are

$$S_i = P_i \text{ (XOR) } C_i$$

$$C_{i+1} = G_i + P_i.C_i$$

where i=0,1,..,3, denotes the bit position being added in FA and the stage in parallel adder

LOOK AHEAD CARRY ADDER

- Using equation for carry-out (C_i) of each stage

$$C_{i+1} = G_i + P_i.C_i$$

The equation for carry-out of each full-adder becomes

$$C_1 = G_0 + P_0.C_0$$

$$C_2 = G_1 + P_1.C_1$$

$$C_3 = G_2 + P_2.C_2$$

$$C_{out} = C_4 = G_3 + P_3.C_3$$

- Substituting the values of C_i in these, we get

$C_1 = G_0 + P_0C_0$

$C_2 = G_1 + P_1C_1$

$= G_1 + P_1 (G_0 + P_0C_0)$

$= G_1 + P_1G_0 + P_1P_0C_0$

$C_3 = G_2 + P_2.C_2$

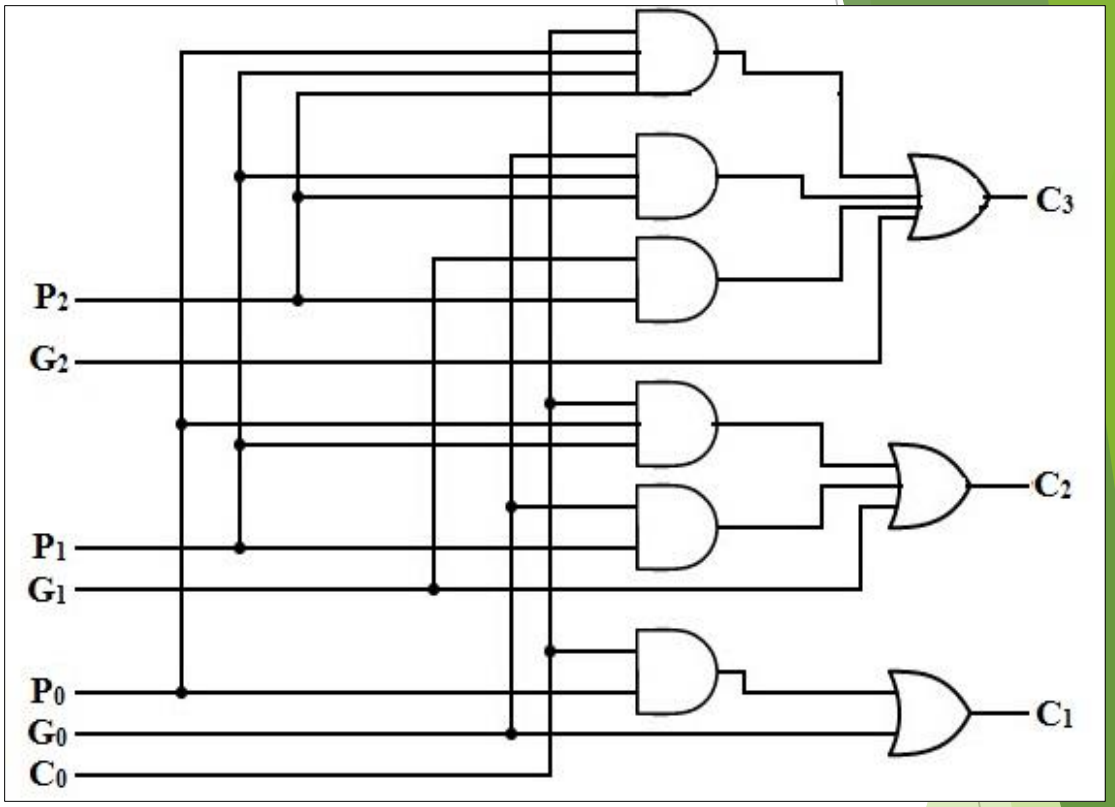
$= G_2 + P_2 (G_1 + P_1G_0 + P_1 P_0C_0)$

$= G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1 P_0C_0$

LOOK AHEAD CARRY ADDER

Logic expressions for each 'C_i' in look ahead carry generator:

$C_1 = G_0 + P_0C_0$
$C_2 = G_1 + P_1G_0 + P_1P_0C_0$
$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$



- It is evident from the equations of C_1, C_2 and C_3, that only one level of AND gate followed by an OR gate will generate each carry-out
- Generation of C_3 does not require C_2 and C_1 to get propagated (as in case of Parallel adder). They all are propagated at the same time

LOOK AHEAD CARRY ADDER

- For all i = {0,1,2,3}, P_i is generated by an XOR gate and G_i by an AND gate
- Carries are propagated through look ahead carry generator and provided as inputs to second XOR gates
- All output carries are generated after delay of two levels of gates
- Each Sum bit requires only two XOR gates
- Sums S_0 - S_3 are generated with equal propagation delays